

Test report

Milestone 3 evidence: "Public GitHub repo(s) ... with **CI logs, coverage report** ..." and "API docs + **test reports with coverage/outcomes**".

The CI run

[Run 36004503722](#), 2026-09-24 13:16 UTC.

GitHub deletes Actions logs after 90 days, so a **full copy lives in the repository**: [ci-run-36004503722.log](#). It is the raw output of `gh run view 36004503722 --log`, with color codes stripped. The only secrets that appear in it are CI test values (JWT_SECRET: ci-secret-jamas-en-produccion, "never in production").

Job	What it runs	Result
App TS	lint (Biome) · typecheck · test ID traceability · tests + coverage on all four TypeScript parts (api, web, shared, cardano) · dependency scan · build · smoke test of the compiled start command	✓
Aiken contracts	aiken fmt --check · aiken check · aiken build, and that the committed plutus.json is up to date	✓
Static analysis	Semgrep, rulesets p/security-audit + p/owasp-top-ten	✓ 128 rules, 536 files, 0 findings
E2E Playwright	the whole app against a migrated and seeded database, on mobile (390 px) and desktop (1440 px) — non-blocking job (continue-on-error)	⚠️ flaky, see below

Results per package

Package	What it covers	Tests	Result
contracts/ (Aiken)	the on-chain validator: stage state machine, thread token, evidence rules	102 (100 unit + 2 property)	✓ 102/102
apps/api	the API: routes, authorization, anchoring, state machine, migrations	712	✓ 709 passed · 3 skipped
packages/cardano	the anchoring port (simulated and real)	115	✓ 115/115
packages/shared	the shared Zod contract, the state machine, Merkle trees	210	✓ 210/210
apps/web (unit)	components, i18n, ApiPort checked against the OpenAPI document	1,593	✓ 1,592 passed · 1 expected it.fails (see below)
apps/web (E2E)	the flows of the four roles plus admin, on two viewports	90	⚠️ flaky, see below
Total (excl. E2E)		2,732	2,728 passed · 3 skipped · 1 expected fail · 0 failed

The 3 skipped tests are intentional: they are `apps/api/test/storage-s3.test.ts`, which runs against a real S3 (MinIO). The same applies to `packages/cardano/src/yaci.test.ts`, which runs against a local Cardano node and is excluded by that package's test script. CI does not start that infrastructure; both are run by hand with `pnpm --filter @plataforma/api test:s3` and `pnpm --filter @plataforma/cardano test:yaci`.

The 1 `it.fails` is intentional: it pins a known, unfixed bug in `LocationMapModal` (`modals-mapa.test.tsx`) — the map may fail to render when the modal opens with `open` set directly rather than through user interaction. It still needs to be reproduced in a real browser before deciding whether to fix the component or the test. **Update 2026-09-28:** reproduced in production (the project-detail location map opened empty) and fixed in the component; the `it.fails` became a regular `it`. The figures in the table above are those of 2026-09-21 and are left as they were.

Coverage

Package	Statements	Branches	Functions	Lines	Threshold	Source
<code>apps/api</code>	100% (2070/2070)	100% (838/838)	100% (424/424)	100% (1900/1900)	99/99/99/99	CI (<code>pnpm test:coverage</code>)
<code>apps/web</code>	100% (1955/1955)	100% (1854/1854)	100% (766/766)	100% (1740/1740)	100/100/100/100	CI (<code>pnpm test:coverage</code>)
<code>packages/shared</code>	100% (247/247)	100% (56/56)	100% (19/19)	100% (231/231)	100/100/100/100	CI (<code>pnpm test:coverage</code>)
<code>packages/cardano</code>	100% (295/295)	100% (149/149)	100% (90/90)	100% (273/273)	95/95/95/95	CI (<code>pnpm test:coverage</code>)

All four TypeScript parts measure 100% on all four metrics, all measured in CI. The SOM's criterion 2 — "*unit tests ≥95% coverage*", read as covering the whole app — is met.

The contracts (`contracts/`) have no line coverage percentage: Aiken does not measure line coverage. The coverage evidence for the validator is [aiken-coverage-report.pdf](#): every condition under which the validator rejects a transaction has at least one test that triggers it.

Security in the same run

- **Semgrep:** 0 findings (see above).
- **Dependency scan (`pnpm audit`):** 0 critical, which is the threshold that blocks the build. The full report lists 12 (4 high, 8 moderate), **all in build and test tooling**, none in what runs in production: `vitest` and its dependencies (`undici`, `nanoid`, `postcss`, `@vitest/mocker`) and the `@tanstack/router-plugin` chain (`browserslist`, `baseline-browser-mapping`). The triage is in the [security review](#).

The E2E job, flaky under CI load (non-blocking)

A separate job, explicitly **non-blocking** (`continue-on-error: true` in `ci.yml`). Running this same suite twice against the same commit: the first run failed 4 of 90 (86 passed), the second failed 5 of 90 (85 passed), with **different tests** failing each time (`admin-certifier-invite`, `panels-DEV-INVESTORS-LIST`, and on the second run also `walkthrough-Walkthrough`) — all Playwright timeouts (`Test timeout of`

30000ms exceeded / expect(locator).toBeVisible() failed), not a wrong business assertion. It is resource contention on GitHub Actions' shared runner (two browsers, mobile and desktop, driving the whole app), not a code defect: **locally, against the same code, the suite runs clean.**

A flaky test, disclosed

One of three local runs of `pnpm test:coverage` failed in `apps/api/test/spec-213-un-bundle-por-stage.test.ts` with `SQLITE_BUSY: database is locked`. The other two passed completely, and so did the CI run. It is contention on the test's SQLite database when coverage instrumentation slows execution down, not a failure of what the test verifies.

How to reproduce

```
pnpm install
pnpm verify:all           # lint + typecheck + tests + build + Aiken
pnpm test:coverage        # coverage of the four TS parts, with their thresholds
pnpm --filter web e2e     # the E2E suite (starts web + api)
```